

Course Project: Data Analysis with Apache Spark

The goal of the ITCS 6190 course project is to **design and implement a big data analytics pipeline using Apache Spark** on a publicly available dataset of your choice, subject to instructor approval. Your project must integrate Structured APIs, Streaming, and MLlib to demonstrate how these components work together to process, analyze, and model large-scale data. You will be responsible for the whole workflow: exploring a dataset, defining meaningful analytical or predictive questions, implementing scalable data processing and real-time streaming ingestion, applying machine learning, and communicating results effectively. All development and submissions will be managed through GitHub, following professional software engineering practices.

Let's have fun exploring data, building cool solutions, and learning Spark together!

1. Team Composition

Each project is completed **individually or in pairs**. Groups of three or more will not be permitted without explicit instructor authorization.

Project Setup Deadline: Students working individually or in pairs must complete their project setup by the end of Week 2. This includes defining their team on Canvas and completing the GitHub setup below. At this milestone, each student or pair is required to:

- Register on *Canvas* as working individually or as a pair.
- Create a private GitHub repository from the provided course template.
- Add your partner (if working in a pair) and the instructor/TAs as collaborators.
- Customize the initial README with member name(s), chosen dataset, and a placeholder project description.

If working in a pair, both members must contribute meaningfully and are expected to understand the complete pipeline, not only the parts they implemented individually. During the live presentation, each student is assessed independently and must be able to speak to every component of the project.

2. Requirements

The course project is designed to give you hands-on experience with the end-to-end development of a big data analytics pipeline using Apache Spark. To ensure that all students

achieve both breadth and depth of learning, your project must meet a defined set of requirements:

- **Dataset** – select a dataset from a publicly available dataset relevant to your analysis goals. Your dataset must be described in the Proposal Issue and approved by the instructor before you proceed. Note that you should store raw or full datasets outside the GitHub repository (e.g., on AWS S3 or via a public URL). Commit only small representative samples for testing.
- **Spark Components** – your pipeline must include:
 - *Structured APIs*: use the DataFrame API for ingestion, transformations, aggregations, and joins.
 - *Spark SQL*: may be used alongside Structured APIs for analysis or transformation. It is not mandatory, but credited when used effectively.
 - *Streaming*: ingest and process real-time or simulated real-time data using Spark Structured Streaming. At a minimum, demonstrate a working streaming job that reads from a source, applies a transformation or aggregation, and outputs results. Production-grade throughput or low-latency guarantees are not required.
 - *MLlib*: apply at least one machine learning task (classification, regression, clustering, or recommendation) using Spark MLlib. At a minimum, train a model on your dataset and report evaluation metrics. Full feature engineering pipelines, hyperparameter tuning, and model comparison are encouraged but not required.
- **Performance Optimization (optional)** – If relevant, demonstrate one or more Spark optimization techniques (e.g., caching, partitioning, broadcast joins, reducing shuffle operations) and explain their impact.
- **Reproducibility** – provide a single command (run.sh or make run) to execute the whole pipeline in local mode, from raw data ingestion through final outputs, ensuring that others can reproduce your results.

3. Timeline and Milestones

This timeline outlines the key milestones for your project, aligned with the course schedule. Milestones are indicative — the table shows the expected pace, but you are encouraged to progress faster when possible. Each week (starting from Week 2) also includes a Weekly

Check-in Issue in GitHub, which is the primary communication channel for progress updates and feedback.

Week	Milestone	To Do
2	Project Setup	Define your project on Canvas (individual or pair), create the GitHub repository from the course template, add your partner (if applicable) and the instructor/TAs as collaborators, and open a Proposal Issue on GitHub with your proposed dataset and analytical questions. The instructor will confirm approval before you proceed to Week 3.
3	Data Ingestion + EDA	Submit a Pull Request with your ingestion code and initial EDA (summaries, sample queries, visualizations). Open a Weekly Check-in Issue summarizing progress and any blockers.
4	Streaming + MLlib	Submit a Pull Request with your streaming job and MLlib model (with evaluation metrics). Open a Weekly Check-in Issue with progress update and next steps.
5	Full Pipeline + Documentation	Submit a Pull Request with the complete, runnable end-to-end pipeline and finalized /docs/ documentation. The pipeline must integrate Structured APIs, Streaming, and MLlib. Spark SQL may be included if used in your analysis. Open a Weekly Check-in Issue confirming readiness for final release.
5 (Jun 22–23)	Final Release + Live Presentation	Create a Git tag (v1.0.0) and a GitHub Release with links to your report and slides. Ensure the pipeline runs with a single command (run.sh or make run). Present and defend your work at the live session.

4. Weekly Check-ins

Each week, starting from Week 2, you are expected to open a **Weekly Check-in Issue** in your GitHub repository. These issues are the primary channel for project communication — all feedback, guidance, and follow-up will be provided directly in the Issue by the instructor. There are no in-class check-in meetings.

Each Weekly Check-in Issue must include:

- **Progress:** summarize what you completed since the previous check-in, with links to relevant commits or Pull Requests.
- **Blockers:** describe any obstacles you are facing (technical, conceptual, or time-related).
- **Plan:** list one or two concrete deliverables you aim to complete before the next check-in.

Guidelines:

- Open the Issue by the end of the week it is due (see milestone table). The instructor will respond with comments, questions, or suggestions directly in the Issue.
- Address any feedback from the previous check-in before opening the next one. Close the Issue once all action items are resolved.
- Use GitHub Issues for all project communication. Do not use email or other channels to request feedback on project work.

5. Deliverables (all in GitHub)

At the end of the project, you will submit a complete package that demonstrates your technical implementation, documents your process, and communicates your results. The following items must be included:

- **Code Notebook/Package** – a complete, runnable implementation of your Spark pipeline. Must include a Makefile or run.sh to allow one-command execution from data ingestion to final output. Code should be clean, well-documented, and organized according to the project structure.
- **Documentation** – located in the /docs/ directory and containing:
 - *Dataset Overview*: source, size, schema, licensing (if any), and preprocessing steps.
 - *Methodology*: detailed description of your processing, modeling, and streaming pipeline.
 - *Results*: key findings, metrics, and visualizations.
 - *Limitations*: known constraints or assumptions in your work.
 - *Reproduction Guide*: clear, step-by-step instructions to reproduce your results.
- **Final Release** – a Git tag (e.g., v1.0.0) with:
 - Release notes summarizing the project.
 - Links to final documentation and presentation.
 - Instructions for running the final pipeline.
- **Presentation** – delivered at the live session, with slides stored in /docs/slides/ or linked in the Final Release. Should cover problem definition, methods, results, key insights, and design decisions.

6. Grading

Your project grade will be based on the following components, with evaluation grounded in concrete evidence from your GitHub repository:

Component	Weight	Evidence in GitHub
Proposal	10%	Well-defined Proposal Issue describing dataset, goals, and planned Spark components, with instructor/TA feedback addressed
Milestones & Check-ins	20%	Timely Pull Requests for each milestone; consistent Weekly Check-in Issues with meaningful progress updates, blockers addressed, and instructor feedback incorporated
Technical Implementation	40%	Functional end-to-end Spark pipeline, correct use of Structured APIs, Streaming, and MLlib, and clean, well-documented code reproducible with a single command
Analysis & Insights	30%	Clear documentation and notebooks presenting methodology, metrics, visualizations, and thoughtful interpretation of results

7. Example Ideas

The following examples illustrate how to design a project that meets all requirements by incorporating Structured APIs, Streaming, and MLlib. Spark SQL may also be included where relevant. You are welcome to use them as inspiration, but are encouraged to propose your own ideas:

- **NYC Taxi Demand & Trip-Time Prediction**
 - *Structured APIs*: ingest and transform large historical trip datasets; join with weather or events data.
 - *Spark SQL*: (optional) aggregate trips by location and time to identify demand hotspots.
 - *Streaming*: simulate real-time trip events (e.g., from live feeds) to update demand estimates.
 - *MLlib*: build a regression model to predict trip duration based on pickup location, time, and traffic indicators.
- **Real-Time Tweet Sentiment Analysis**
 - *Structured APIs*: clean and tokenize tweets, join with a sentiment lexicon.
 - *Spark SQL*: (optional) run keyword-based sentiment summaries per time window or hashtag.
 - *Streaming*: ingest tweets from a live API or simulated feed in micro-batches.
 - *MLlib*: train a classification model to predict sentiment labels from tweet text features.

- **MovieLens Recommendation System**

- *Structured APIs*: process and join user–movie ratings with metadata such as genres.
- *Spark SQL*: (optional) query top-rated movies per genre or user segment.
- *Streaming*: simulate live rating events and update recommendations on the fly.
- *MLlib*: apply collaborative filtering (ALS) to generate personalized movie recommendations.

- **IoT Sensor Anomaly Detection**

- *Structured APIs*: preprocess raw sensor readings and compute derived metrics.
- *Spark SQL*: (optional) aggregate readings over time windows to detect unusual patterns.
- *Streaming*: continuously ingest simulated sensor data in real time.
- *MLlib*: apply clustering or anomaly detection algorithms to identify abnormal behavior.

8. Tips for Success

A strong process is just as crucial as strong technical skills. The following practices will help you work efficiently and deliver a high-quality project:

- **Plan before you code** – spend time in week 1 understanding your dataset thoroughly before writing pipeline code. Good analytical questions make the whole project more coherent.
- **Choose a dataset you care about** – interest in the topic will keep you motivated and make the analysis more insightful.
- **Work transparently** – use GitHub Issues and Pull Requests for all work so progress is visible, and no code is hidden off-platform.
- **Break work into small milestones** – aim for frequent, incremental progress and merge changes early to avoid large, unmanageable PRs.
- **Use Weekly Check-in Issues actively** – don't wait until you're stuck. Opening an Issue early with a clear blocker gets you faster feedback and prevents small problems from becoming large ones.